

Ghid complet PAOJ pentru examen

Java - explicații, capcane, exemple și exerciții

Cursurile 01-14: Java de bază, referințe, clase, moștenire, string-uri, interfețe, excepții, fișiere, serializare, JDBC, colecții, lambda, stream-uri, fire de executare și socket-uri

Acest material este făcut ca recapitulare rapidă pentru examen: explică ideile din spate, arată ce se întâmplă în memorie, scoate în evidență capcanele și oferă exerciții de tip „ce se afișează?” / „se compilează?”.

Cuprins

Cum să folosești ghidul	4
1 Introducere în Java	5
1.1 JVM, JRE, JDK și JIT	5
1.2 Metoda main	5
1.3 Primitive vs referințe	6
1.4 Stack vs heap	6
1.5 Inițializare implicită	6
1.6 Literali și conversii	6
1.7 Promovare numerică	7
1.8 Operatori și scurtcircuit	7
1.9 Mini-exerciții	7
2 Referințe, tablouri, clase și obiecte	8
2.1 Referință vs obiect	8
2.2 null	8
2.3 Tablouri	8
2.4 Enhanced for	9
2.5 Tablouri bidimensionale	9
2.6 Clasa Arrays	9
2.7 Constructori	10
2.8 this și transmiterea parametrilor	10
2.9 static și final	10
2.10 Shallow copy vs deep copy	11
2.11 Blocuri de inițializare	11
3 Clase, obiecte, moștenire și polimorfism	12
3.1 Moștenirea	12
3.2 Ce se moștenește	12
3.3 Constructorii în moștenire	12
3.4 Overriding vs overloading	13
3.5 Regula de aur: câmpuri, metode statice, metode non-statice	13
3.6 Upcasting, downcasting, instanceof	13
3.7 Object: toString, equals, hashCode	14
3.8 Clase abstracte	14
4 Agregare, compoziție, String, imutabilitate și record	15
4.1 IS_A vs HAS_A	15
4.2 Agregare vs compoziție	15
4.3 String literal pool	15
4.4 intern	15
4.5 String este imutabil	16
4.6 Metode importante String	16
4.7 StringBuilder și StringBuffer	16

4.8	Clase imutabile	17
4.9	Records	17
5	Interfețe I	18
5.1	Ce este o interfață	18
5.2	Reguli implicite	18
5.3	Referință de tip interfață	18
5.4	Mai multe interfețe	19
5.5	Comparable și Comparator	19
5.6	Callback	19
6	Interfețe II și enumerări	20
6.1	Interfețe marker	20
6.2	Cloneable și clone	20
6.3	Shallow clone vs deep clone	20
6.4	Metode default, static, private în interfețe	20
6.5	Conflict între default-uri	21
6.6	sealed	21
6.7	Enum	21
7	Excepții	22
7.1	Ierarhia excepțiilor	22
7.2	try-catch-finally	22
7.3	Ordinea catch-urilor	22
7.4	finally	22
7.5	throw vs throws	23
7.6	Excepții proprii	23
7.7	try-with-resources	23
8	Fișiere și fluxuri I/O	24
8.1	Fluxuri	24
8.2	File	24
8.3	FileReader / FileWriter	24
8.4	Append vs overwrite	24
8.5	BufferedReader / BufferedWriter	24
8.6	DataInputStream / DataOutputStream	25
8.7	RandomAccessFile	25
9	Serializare și externalizare	26
9.1	Serializare	26
9.2	Ce se serializează	26
9.3	NotSerializableException	27
9.4	Constructor la deserializare	27
9.5	serialVersionUID	27
9.6	Externalizare	27
10	JDBC și colecții	28
10.1	JDBC pe scurt	28
10.2	Ierarhia colecțiilor	28
10.3	List	28
10.4	Set	29
10.5	Map	29
10.6	Iterator	29

11 Programare funcțională	30
11.1 Imperativ vs funcțional	30
11.2 Lambda expresii	30
11.3 Interfață funcțională	30
11.4 Descriptori funcționali	30
11.5 Predicate	31
11.6 Consumer	31
11.7 Function	31
11.8 Supplier	31
11.9 Method references	31
11.10 Variabile efectiv finale	32
12 Stream-uri	33
12.1 Ce este un stream	33
12.2 Creare stream	33
12.3 Operații intermediare	33
12.4 map vs flatMap	34
12.5 Operații terminale	34
12.6 reduce	34
12.7 collect și Collectors	34
12.8 Capcane la stream-uri	35
13 Fire de executare și socket-uri	36
13.1 Proces vs thread	36
13.2 Crearea thread-urilor	36
13.3 start vs run	36
13.4 join, sleep, interrupt, volatile	37
13.5 synchronized	37
13.6 wait, notify, notifyAll	37
13.7 Socket-uri	37
14 Test final de recapitulare	39
14.1 Exerciții rapide	39
14.2 Checklist final înainte de examen	40
Asocieri rapide de memorat	41

Cum să folosești ghidul

Reține

Pentru grile, nu înveța doar definiții. La Java contează să urmărești: tipul declarat vs tipul real, obiect vs referință, compilare vs rulare, metode statice vs non-statice, checked vs unchecked, operație intermediară vs terminală.

Ordinea recomandată de recapitulare

1. Începe cu regulile de bază: primitive/referințe, stack/heap, static, final, main.
2. Continuă cu OOP: constructori, moștenire, overriding, polimorfism, equals/hashCode.
3. Repetă bine interfețe, Comparable, Comparator, lambda și stream-uri.
4. Finalizează cu excepții, fișiere, serializare, colecții, thread-uri.
5. La fiecare capitol, rezolvă exemplele fără să te uiți întâi la răspuns.

Capitolul 1 Introducere în Java

1.1 JVM, JRE, JDK și JIT

Important

Java este compilat în bytecode, nu direct în cod mașină. Bytecode-ul este executat de JVM.

Fluxul este:

Test.java -> javac -> Test.class -> JVM -> execuție

Termen	Sens
JVM	Java Virtual Machine. Execută bytecode-ul.
JRE	JVM + biblioteci necesare rulării programelor Java.
JDK	JRE + unelte pentru programator, de exemplu javac.
JIT	Just-In-Time compiler. Compilează în timpul rulării bucăți de bytecode folosite des în cod mașină nativ.

Reține

JVM rulează programul. JIT accelerează programul. JDK este pentru dezvoltare, JRE pentru rulare.

1.2 Metoda main

Forma corectă:

```
public static void main(String[] args) {  
    // cod  
}
```

main trebuie să fie static deoarece JVM trebuie să pornească programul fără să creeze mai întâi un obiect.

Capcană de examen

```
public void main(String[] args) { }
```

Nu este punct de pornire valid. Lipsește static.

1.3 Primitive vs referințe

Primitive	byte, short, int, long, float, double, char, boolean. Variabila ține direct valoarea.
Referințe	Clase, tablouri, interfețe, enum-uri. Variabila ține o referință către un obiect din heap.

```
int x = 10;  
String s = new String("Ana");
```

Mental:

- x conține direct valoarea 10;
- s conține o referință spre obiectul String din heap.

1.4 Stack vs heap

Stack	Variabile locale, parametri, apeluri de metode. Fiecare thread are stack propriu.
Heap	Obiecte create cu new. Este comun pentru thread-urile aceluiasi proces.

1.5 Inițializare implicită

Important

Variabilele locale nu primesc valori implicite. Datele membre primesc valori implicite.

```
class A {  
    int x;           // 0  
    boolean ok;      // false  
    String s;        // null  
  
    public static void main(String[] args) {  
        int y;  
        System.out.println(y); // eroare de compilare  
    }  
}
```

1.6 Literalii și conversii

```
int a = 10;           // baza 10  
int b = 0b10;         // baza 2 -> 2  
int c = 012;          // baza 8 -> 10  
int d = 0x10;         // baza 16 -> 16  
long e = 10L;         // long  
float f = 3.14f;      // float
```

Capcană de examen

012 nu este doisprezece. Este literal octal, deci valoarea este 10 în baza 10.

1.7 Promovare numerică

La operații aritmetice, byte, short și char sunt promovate la int.

```
byte b = 1;
b = b + 1;           // eroare: b + 1 este int
b = (byte)(b+1);    // corect
b += 1;             // corect, include cast implicit
```

```
char c = 'A';
System.out.println(c + 1);           // 66
System.out.println((char)(c+1));    // B
```

1.8 Operatori și scurtcircuit

```
int x = 0;
if (x != 0 && 10 / x > 1) {
    System.out.println("ok");
}
```

Nu apare eroare, pentru că && nu mai evaluează a doua condiție dacă prima este falsă.

Capcană de examen

& pe boolean evaluează ambele părți. && face scurtcircuit.

1.9 Mini-exerciții

Exemplu / exercițiu

```
System.out.println(0b10 + 010 + 0x10);
```

Răspuns: 2 + 8 + 16 = 26.

Exemplu / exercițiu

```
float f = 2.5;
```

Nu compilează. 2.5 este double. Corect: 2.5f.

Capitolul 2 Referințe, tablouri, clase și obiecte

2.1 Referință vs obiect

```
Persoana p = new Persoana();  
Persoana q = p;
```

Aici nu ai două obiecte. Ai două referințe către același obiect.

```
class Persoana {  
    String nume;  
}  
  
Persoana p = new Persoana();  
p.nume = "Ana";  
Persoana q = p;  
q.nume = "Maria";  
System.out.println(p.nume); // Maria
```

2.2 null

null înseamnă că referința nu indică niciun obiect.

```
String s = null;  
System.out.println(s instanceof String); // false  
System.out.println(s.length());          // NullPointerException
```

Reține

instanceof cu null dă false, nu excepție. Excepția apare când accesezi metodă/câmp prin null.

2.3 Tablouri

Tablourile sunt obiecte, deci sunt tipuri de referință.

```
int[] v = new int[3];  
System.out.println(v[0]); // 0  
System.out.println(v.length); // 3
```

La tablou ai length, nu length().

Capcană de examen

```
int a[], b;
```

Aici a este tablou, dar b este int. Recomandat: `int[] a, b;`.

2.4 Enhanced for

```
int[] v = {1, 2, 3};
for (int x : v) {
    x = 100;
}
System.out.println(Arrays.toString(v)); // [1, 2, 3]
```

Variabila x este copie a elementului, nu elementul din tablou.

2.5 Tablouri bidimensionale

În Java, un tablou 2D este un tablou de referințe către alte tablouri. Liniile pot avea lungimi diferite.

```
int[][] a = {{1, 2}, {3}, {4, 5, 6}};
System.out.println(a.length); // 3
System.out.println(a[1].length); // 1
```

Parcursere corectă:

```
for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < a[i].length; j++) {
        System.out.print(a[i][j] + " ");
    }
}
```

2.6 Clasa Arrays

Metodă	Ce face
toString	Afișează elementele unui tablou 1D.
deepToString	Afișează conținutul tablourilor multidimensionale.
equals	Compară elementele pentru tablouri 1D; la 2D compară referințele liniilor.
deepEquals	Compară în profunzime tablouri multidimensionale.
mismatch	Primul index unde diferă sau -1 dacă sunt egale.
compare	Comparare lexicografică.
copyOf	Copiază, trunchiază sau completează cu valori implicite.
binarySearch	Caută în tablou sortat. Dacă nu găsește: -pozitieInserare - 1.

```
int[][] a = {{1}, {2}};
int[][] b = {{1}, {2}};
System.out.println(Arrays.equals(a, b)); // false
System.out.println(Arrays.deepEquals(a, b)); // true
```

2.7 Constructori

Constructorul are același nume cu clasa și nu are tip returnat.

```
class Persoana {
    String nume;

    Persoana() { // acces default / package-private
        nume = "Anonim";
    }

    Persoana(String nume) {
        this.nume = nume;
    }
}
```

Capcană de examen

Dacă scrii un constructor cu parametri, Java nu mai generează constructor fără parametri.

```
class A {
    A(int x) {}
}
A a = new A(); // eroare
```

2.8 this și transmiterea parametrilor

Java transmite parametrii întotdeauna prin valoare. Pentru referințe, se copiază valoarea referinței.

```
static void f(int[] v) {
    v[0] = 100;
    v = new int[]{7, 8, 9};
    v[1] = 200;
}

int[] a = {1, 2, 3};
f(a);
System.out.println(Arrays.toString(a)); // [100, 2, 3]
```

2.9 static și final

static aparține clasei, nu obiectului.

```
class A {
    static int x;
}
A a = new A();
A b = new A();
a.x = 10;
b.x = 20;
System.out.println(a.x); // 20
```

final la referințe blochează referința, nu obiectul.

```
final int[] v = {1, 2, 3};  
v[0] = 100;      // corect  
v = new int[10]; // eroare
```

2.10 Shallow copy vs deep copy

```
class A {  
    int[] v;  
    A(A ob) {  
        this.v = ob.v; // shallow copy  
    }  
}
```

Shallow copy copiază referința. Deep copy creează obiect nou intern.

```
class A {  
    int[] v;  
    A(A ob) {  
        this.v = Arrays.copyOf(ob.v, ob.v.length); // deep copy  
    }  
}
```

2.11 Blocuri de inițializare

```
class A {  
    static { System.out.println("static"); }  
    { System.out.println("nstatic"); }  
    A() { System.out.println("constructor"); }  
}  
new A();  
new A();
```

Output:

```
static  
nstatic  
constructor  
nstatic  
constructor
```

Capitolul 3 Clase, obiecte, moștenire și polimorfism

3.1 Moștenirea

Moștenirea este relație IS_A. O subclasă extinde o superclasă.

```
class Student extends Persoana {  
    String facultate;  
}
```

Java permite moștenire simplă între clase: o clasă poate extinde o singură clasă directă.

3.2 Ce se moștenește

- Membrii public și protected se moștenesc.
- Membrii private există în obiect, dar nu pot fi accesați direct din subclasă.
- Constructorii nu se moștenesc.
- Toate clasele moștenesc direct sau indirect `Object`.

3.3 Constructorii în moștenire

La crearea unui obiect de tip subclasă, se execută întâi constructorul superclasei, apoi constructorul subclasei.

```
class A {  
    A() { System.out.print("A "); }  
}  
class B extends A {  
    B() { System.out.print("B "); }  
}  
new B(); // A B
```

Capcană de examen

Dacă superclasa nu are constructor fără parametri, subclasa trebuie să apeleze explicit `super(...)`.

```
class A { A(int x) {} }  
class B extends A {  
    B() { super(10); }  
}
```

3.4 Overriding vs overloading

Criteriu	Overloading	Overriding
Traducere	supraîncărcare	suprascrisiere
Parametri	diferiți	identici
Unde	aceeași clasă sau moștenire	în subclasă
Return type	nu diferențiază singur	același sau covariant
Moment decizie	compilare	rulare

```
class A {
    void f() { System.out.println("A"); }
}
class B extends A {
    @Override
    void f() { System.out.println("B"); }
}
A ob = new B();
ob.f(); // B
```

3.5 Regula de aur: câmpuri, metode statice, metode non-statice

Important

Câmpurile se aleg după tipul declarat. Metodele statice se aleg după tipul declarat. Metodele non-statice suprascrise se aleg după tipul real al obiectului.

```
class A {
    int x = 1;
    void f() { System.out.println("A"); }
    static void g() { System.out.println("static A"); }
}
class B extends A {
    int x = 2;
    void f() { System.out.println("B"); }
    static void g() { System.out.println("static B"); }
}
A ob = new B();
System.out.println(ob.x); // 1
ob.f(); // B
ob.g(); // static A
```

3.6 Upcasting, downcasting, instanceof

```
A a = new B(); // upcasting implicit
B b = (B) a; // downcasting explicit
```

Downcasting-ul greșit compilează, dar poate da `ClassCastException` la rulare.

```
Angajat a = new Economist();  
Inginer i = (Inginer) a; // ClassCastException
```

Sigur:

```
if (a instanceof Inginer) {  
    Inginer i = (Inginer) a;  
}
```

3.7 Object: toString, equals, hashCode

toString() este apelată automat de System.out.println(ob).

```
@Override  
public String toString() {  
    return nume + " " + varsta;  
}
```

== compară referințele. equals() ar trebui să compare conținutul.

```
@Override  
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (obj == null || getClass() != obj.getClass()) return false;  
    Persoana p = (Persoana) obj;  
    return varsta == p.varsta && Objects.equals(nume, p.nume);  
}  
  
@Override  
public int hashCode() {  
    return Objects.hash(nume, varsta);  
}
```

Reține

Dacă equals() este true, atunci hashCode() trebuie să fie egal. Invers nu este obligatoriu: pot exista coliziuni.

3.8 Clase abstracte

O clasă abstractă nu poate fi instanțiată, dar poate fi tip de referință.

```
abstract class Angajat {  
    abstract double calculSalariu();  
}  
class Paznic extends Angajat {  
    double calculSalariu() { return 3000; }  
}  
Angajat a = new Paznic(); // corect
```

O clasă concretă care extinde o clasă abstractă trebuie să implementeze metodele abstracte.

Capitolul 4 Agregare, compoziție, String, imutabilitate și record

4.1 IS_A vs HAS_A

Relație	Exemplu	Sens
IS_A	Student extends Persoana	Student este Persoană.
HAS_A CAN_DO	House has Room implements Runnable	Obiectul conține alt obiect. Obiectul poate face ceva.

4.2 Agregare vs compoziție

Concept	Sens	Cod tipic
Agregare	Obiectul intern poate exista independent.	<code>this.x = x;</code>
Compoziție	Containerul controlează obiectul intern.	<code>this.x = new X(x);</code>

4.3 String literal pool

```
String a = "java";  
String b = "java";  
String c = new String("java");  
  
System.out.println(a == b);    // true  
System.out.println(a == c);    // false  
System.out.println(a.equals(c)); // true
```

Reține

`==` compară referințe. `equals()` compară conținut la String.

4.4 intern


```
String a = "abc";
String b = new String("abc");
System.out.println(a == b); // false
b = b.intern();
System.out.println(a == b); // true
```

4.5 String este imutabil

```
String s = "abc";
s.toUpperCase();
System.out.println(s); // abc

s = s.toUpperCase();
System.out.println(s); // ABC
```

Important

Metode precum `toUpperCase()`, `substring()`, `replaceAll()` returnează un `String` nou. Nu modifică obiectul vechi.

Rezultatul obținut la runtime nu intră automat în string literal pool. Pentru asta există `intern()`.

4.6 Metode importante String

Metodă	Sens
<code>length()</code>	Lungimea șirului. La tablou este <code>length</code> , fără paranteze.
<code>charAt(i)</code>	Caracterul de pe poziția <code>i</code> .
<code>substring(a,b)</code>	De la <code>a</code> inclusiv până la <code>b</code> exclusiv.
<code>indexOf</code>	Prima poziție sau -1.
<code>lastIndexOf</code>	Ultima poziție sau -1.
<code>compareTo</code>	Comparare lexicografică. Contează semnul.
<code>matches</code>	Verifică dacă tot șirul respectă regex-ul.
<code>replaceAll</code>	Înlocuiește toate potrivirile regex.
<code>split</code>	Sparge șirul după regex.

4.7 StringBuilder și StringBuffer

Clasă	Mutabilă	Thread-safe	Viteză
<code>String</code>	Nu	Da prin imutabilitate	bună pentru valori fixe
<code>StringBuilder</code>	Da	Nu	rapidă
<code>StringBuffer</code>	Da	Da	mai lentă

```
StringBuilder sb = new StringBuilder("abc");
sb.append("d");
sb.setCharAt(0, 'X');
System.out.println(sb); // Xbcd
```

4.8 Clase imutabile

Reguli:

1. clasa `final`;
2. câmpuri `private final`;
3. fără setteri;
4. copii defensive pentru obiecte mutabile;
5. getterii nu returnează referințe interne mutabile.

4.9 Records

```
record Student(String nume, int grupa, double medie) {}
```

Compilerul generează:

- câmpuri `private final`;
- constructor canonic;
- metode de acces: `nume()`, `grupa()`, `medie()`;
- `equals()`, `hashCode()`, `toString()`.

Capcană de examen

Record-urile sunt shallow immutable. Dacă un câmp este un obiect mutabil, trebuie făcută copie defensivă în constructor și în accessor.

Capitolul 5 Interfețe I

5.1 Ce este o interfață

O interfață este un contract de comportament. O clasă care o implementează promite că oferă metodele abstracte cerute.

```
interface Platibil {  
    void plateste(double suma);  
}  
class Persoana implements Platibil {  
    public void plateste(double suma) {  
        System.out.println("Plata: " + suma);  
    }  
}
```

5.2 Reguli implicite

În interfață:

- câmpurile sunt implicit `public static final`;
- metodele fără corp sunt implicit `public abstract`;
- metodele implementate în clasă trebuie să fie `public`.

Capcană de examen

```
interface I { void f(); }  
class A implements I {  
    void f() {} // eroare, acces default este mai restrictiv  
}
```

Corect: `public void f() {}`.

5.3 Referință de tip interfață

```
interface I { void f(); }  
class A implements I {  
    public void f() {}  
    public void g() {}  
}  
I ob = new A();  
ob.f(); // corect  
ob.g(); // eroare
```

Referința de tip interfață vede doar metodele din interfață.

5.4 Mai multe interfețe

O clasă poate implementa mai multe interfețe.

```
class A implements I1, I2, I3 { }
```

Dacă două interfețe au aceeași metodă abstractă cu aceeași semnătură și același return, o singură implementare este suficientă. Dacă return type diferă, nu se poate.

5.5 Comparable și Comparator

Comparable definește ordinea naturală în clasă.

```
class Persoana implements Comparable<Persoana> {  
    String nume;  
    public int compareTo(Persoana p) {  
        return this.nume.compareTo(p.nume);  
    }  
}
```

Comparator definește un criteriu extern.

```
Comparator<Persoana> dupaVarsta = (p1, p2) -> p1.getVarsta() - p2.getVarsta();  
Arrays.sort(v, dupaVarsta);
```

Regula compareTo/compare:

- negativ: primul vine înainte;
- zero: egale ca ordine;
- pozitiv: primul vine după.

5.6 Callback

Callback = trimiți comportament ca parametru.

```
interface FunctieGenerica {  
    int functie(int x);  
}  
  
static int calculeazaSuma(FunctieGenerica fg, int n) {  
    int s = 0;  
    for (int i = 1; i <= n; i++) s += fg.functie(i);  
    return s;  
}
```

Apeluri:

```
calculeazaSuma(x -> x, 10);  
calculeazaSuma(x -> x * x, 10);
```

Capitolul 6 Interfețe II și enumerări

6.1 Interfețe marker

O interfață marker este o interfață goală, folosită ca etichetă.

Exemple:

- Serializable;
- Cloneable.

6.2 Cloneable și clone

```
class A implements Cloneable {  
    public Object clone() throws CloneNotSupportedException {  
        return super.clone();  
    }  
}
```

Capcană de examen

Dacă nu implementezi Cloneable, apelul `super.clone()` poate arunca `CloneNotSupportedException`.

6.3 Shallow clone vs deep clone

`super.clone()` face shallow copy: copiază referințele interne. Pentru deep clone trebuie clonate și obiectele interne.

6.4 Metode default, static, private în interfețe

```
interface I {  
    default void f() { System.out.println("default"); }  
    static void g() { System.out.println("static"); }  
    private void h() { System.out.println("helper"); }  
}
```

- default: clasa poate moșteni sau suprascrie.
- static: se apelează cu `I.g()`, nu prin obiect.
- private: doar cod comun intern în interfață.

6.5 Conflict între default-uri

```
interface I1 { default void f() {} }
interface I2 { default void f() {} }
class A implements I1, I2 { } // eroare
```

Trebuie rezolvat explicit:

```
class A implements I1, I2 {
    public void f() {
        I1.super.f();
    }
}
```

6.6 sealed

sealed controlează cine poate extinde o clasă sau implementa o interfață.

```
sealed class Angajat permits Economist, Paznic { }
final class Paznic extends Angajat { }
non-sealed class Economist extends Angajat { }
```

Subclasa permisă trebuie să fie final, sealed sau non-sealed.

6.7 Enum

```
enum Zi {
    LUNI, MARTI, MIERCURI
}
System.out.println(Zi.LUNI.name()); // LUNI
System.out.println(Zi.LUNI.ordinal()); // 0
```

Reține

Enum-ul nu se instanțiază cu new. Constructorul enum este privat implicit. ordinal() pornește de la 0.

Enum cu valori:

```
enum Directie {
    NORD(1, 'N'), EST(2, 'E');
    private final int cod;
    private final char simbol;
    Directie(int cod, char simbol) {
        this.cod = cod;
        this.simbol = simbol;
    }
}
```

Capitolul 7 Excepții

7.1 Ierarhia excepțiilor

Throwable -> Error / Exception -> RuntimeException

Tip	Sens
Error	Probleme grave ale JVM. De obicei nu le tratezi.
Checked exception	Compilatorul cere try-catch sau throws. Exemplu: IOException.
Unchecked exception	Subclase din RuntimeException. Nu ești obligat să le tratezi. Exemplu: NullPointerException.

7.2 try-catch-finally

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("impartire la zero");  
} finally {  
    System.out.println("finally");  
}
```

7.3 Ordinea catch-urilor

Catch-urile se pun de la particular la general.

```
try { }  
catch (FileNotFoundException e) { }  
catch (IOException e) { }  
catch (Exception e) { }
```

Capcană de examen

```
catch (Exception e) { }  
catch (IOException e) { } // unreachable
```

Nu compilează.

7.4 finally

finally se execută aproape mereu, chiar și cu return.

```
static int f() {  
    try {  
        return 1;  
    } finally {  
        System.out.println("finally");  
    }  
}
```

Se afișează finally, apoi se returnează 1.

7.5 throw vs throws

throw	Aruncă efectiv o excepție.
throws	Declară în antet că metoda poate arunca excepția.

```
void f() throws IOException {  
    throw new IOException("eroare");  
}
```

7.6 Excepții proprii

Checked:

```
class MyException extends Exception {  
    MyException(String mesaj) { super(mesaj); }  
}
```

Unchecked:

```
class MyException extends RuntimeException {  
    MyException(String mesaj) { super(mesaj); }  
}
```

7.7 try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader("date.txt"))) {  
    String linie = br.readLine();  
} catch (IOException e) {  
    System.out.println("Eroare IO");  
}
```

Resursa se închide automat dacă implementează AutoCloseable.

Capitolul 8 Fișiere și fluxuri I/O

8.1 Fluxuri

Un flux este un canal de citire/scriere.

Caracter	Reader/Writer. Pentru fișiere text.
Octet	InputStream/OutputStream. Pentru fișiere binare.
Primitiv	Citește/scrie direct din sursă/destinație.
Procesare	Învelește alt flux și adaugă funcționalități.

8.2 File

File reprezintă fișier/director, dar nu citește și nu scrie conținut.

```
File f = new File("date.txt");
System.out.println(f.getName());
System.out.println(f.getAbsolutePath());
```

8.3 FileReader / FileWriter

```
try (FileReader fin = new FileReader("in.txt");
    FileWriter fout = new FileWriter("out.txt")) {
    int c;
    while ((c = fin.read()) != -1) {
        fout.write(c);
    }
}
```

read() returnează int, pentru că trebuie să poată returna -1 la final de fișier.

8.4 Append vs overwrite

```
new FileWriter("out.txt", true); // append
new FileWriter("out.txt", false); // overwrite
new FileWriter("out.txt"); // overwrite
```

8.5 BufferedReader / BufferedWriter

```
try (BufferedReader br = new BufferedReader(new FileReader("date.in"));
    BufferedWriter bw = new BufferedWriter(new FileWriter("date.out"))) {
    String linie;
    while ((linie = br.readLine()) != null) {
        bw.write(linie);
        bw.newLine();
    }
}
```

readLine() returnează null la final de fișier.

8.6 DataInputStream / DataOutputStream

Acestea sunt fluxuri de procesare peste fluxuri de octeți.

```
DataOutputStream out = new DataOutputStream(
    new FileOutputStream("numere.bin")
);
out.writeInt(10);
out.writeDouble(2.5);
```

Citirea trebuie să respecte aceeași ordine:

```
DataInputStream in = new DataInputStream(
    new FileInputStream("numere.bin")
);
int a = in.readInt();
double b = in.readDouble();
```

Capcană de examen

Dacă scrii int, double, dar citești double, int, interpretezi greșit bytes.

8.7 RandomAccessFile

```
RandomAccessFile raf = new RandomAccessFile("fisier.bin", "rw");
raf.seek(10);
raf.writeInt(100);
System.out.println(raf.getFilePointer()); // 14
```

Un int ocupă 4 octeți.

Capitolul 9 Serializare și externalizare

9.1 Serializare

Serializarea transformă obiectul în bytes. Deserializarea reconstruiește obiectul din bytes.

```
class Student implements Serializable {  
    private static final long serialVersionUID = 1L;  
    String nume;  
    int grupa;  
}
```

Scriere:

```
try (ObjectOutputStream out = new ObjectOutputStream(  
    new FileOutputStream("student.bin"))) {  
    out.writeObject(new Student("Ana", 232));  
}
```

Citire:

```
try (ObjectInputStream in = new ObjectInputStream(  
    new FileInputStream("student.bin"))) {  
    Student s = (Student) in.readObject();  
}
```

9.2 Ce se serializează

- datele membre de instanță;
- inclusiv private;
- graful de obiecte referite, dacă și ele sunt serializabile.

Nu se serializează automat:

- câmpurile static;
- câmpurile transient.

```
class User implements Serializable {  
    static String rol = "admin";  
    String username = "andra";  
    transient String parola = "1234";  
}
```

După deserializare, username este refăcut, parola devine null, iar rol nu vine din fișier.

9.3 NotSerializableException

Dacă un obiect intern nu implementează `Serializable`, apare `NotSerializableException`.

```
class Facultate { }  
class Student implements Serializable {  
    Facultate f = new Facultate();  
}  
// writeObject(new Student()) -> NotSerializableException
```

9.4 Constructor la deserializare

Constructorul clasei serializabile nu se apelează normal la deserializare. Obiectul este reconstruit din bytes.

9.5 serialVersionUID

`serialVersionUID` fixează versiunea clasei. Dacă versiunea clasei din fișier nu se potrivește cu versiunea curentă, poate apărea `InvalidClassException`.

9.6 Externalizare

`Externalizable` îți dă control manual: alegi exact ce scrii și ce citești.

```
class Student implements Externalizable {  
    String nume;  
    int grupa;  
  
    public Student() {} // obligatoriu  
  
    public void writeExternal(ObjectOutput out) throws IOException {  
        out.writeUTF(nume);  
        out.writeInt(grupa);  
    }  
  
    public void readExternal(ObjectInput in)  
        throws IOException, ClassNotFoundException {  
        nume = in.readUTF();  
        grupa = in.readInt();  
    }  
}
```

Reține

La externalizare, ordinea scrierii trebuie să fie identică cu ordinea citirii.

Capitolul 10 JDBC și colecții

10.1 JDBC pe scurt

DriverManager	Obține conexiunea la baza de date.
Connection	Conexiunea activă.
Statement	SQL simplu.
PreparedStatement	SQL cu parametri ?.
CallableStatement	Proceduri/funcții stocate.
ResultSet	Rezultatul unui SELECT.

```
Connection con = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = con.prepareStatement(
    "UPDATE persoane SET nume = ? WHERE cod = ?"
);
ps.setString(1, "Ana");
ps.setInt(2, 5);
ps.executeUpdate();
```

executeQuery() pentru SELECT. executeUpdate() pentru
INSERT/UPDATE/DELETE/DDL.

10.2 Ierarhia colecțiilor

Iterable -> Collection -> List / Set / Queue

Important

Map nu extinde Collection. Se parcurge prin entrySet(), keySet() sau values().

10.3 List

List este ordonată și permite duplicate.

```
List<Integer> l = new ArrayList<>();
l.add(1);
l.add(2);
l.add(1, 5);
System.out.println(l); // [1, 5, 2]
```

ArrayList	Tablou dinamic. Rapid la get(i). Inserarea/ștergerea în mijloc costă.
-----------	---

LinkedList	Listă dublu înlănțuită. Bună la operații la capete, slabă la acces după index.
------------	--

10.4 Set

Set nu permite duplicate.

HashSet	Fără ordine garantată. Folosește hashCode() și equals().
LinkedHashSet	Păstrează ordinea de inserare.
TreeSet	Sortează elementele. Folosește compareTo() sau Comparator.

Capcană de examen

TreeSet consideră duplicate elementele pentru care comparația întoarce 0. HashSet se bazează pe equals/hashCode.

10.5 Map

Map ține perechi cheie-valoare.

```
Map<String, Integer> m = new HashMap<>();
System.out.println(m.put("a", 10)); // null
System.out.println(m.put("a", 20)); // 10
System.out.println(m);              // {a=20}
```

HashMap	Fără ordine garantată. Folosește hash-ul cheii. Permite o cheie null.
TreeMap	Sortează după chei. Folosește compareTo sau Comparator.
LinkedHashMap	Păstrează ordinea de inserare.

Parcurgere:

```
for (Map.Entry<String, Integer> e : m.entrySet()) {
    System.out.println(e.getKey() + " " + e.getValue());
}
```

10.6 Iterator

```
Iterator<String> it = lista.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Capitolul 11 Programare funcțională

11.1 Imperativ vs funcțional

Imperativ: spui pașii. Funcțional/declarativ: spui transformarea dorită.

```
int s = 0;
for (int i = 1; i <= n; i++) s += i;
```

```
int s = IntStream.rangeClosed(1, n).sum();
```

11.2 Lambda expresii

Sintaxa:

(parametri) -> expresie sau (parametri) -> { instrucțiuni }

```
(int a, int b) -> a + b
(a, b) -> a + b
x -> x * x
() -> System.out.println("Hello")
```

Capcană de examen

Dacă folosești acolade și lambda trebuie să returneze valoare, ai nevoie de return.

```
x -> { return x * x; }
```

11.3 Interfață funcțională

O interfață funcțională are o singură metodă abstractă.

```
interface FunctieGenerica {
    int functie(int x);
}
FunctieGenerica f = x -> x * x;
System.out.println(f.functie(5)); // 25
```

11.4 Descriptori funcționali

Interfață	Metodă	Formă	Exemplu
-----------	--------	-------	---------

Predicate<T>	test	T -> boolean	p -> p.getVarsta() >= 30
Consumer<T>	accept	T -> void	s -> System.out.println(s)
Function<T,R>	apply	T -> R	p -> p.getNume()
Supplier<R>	get	() -> R	() -> new Persoana()

11.5 Predicate

```
Predicate<Persoana> adult = p -> p.getVarsta() >= 18;
Predicate<Persoana> numeA = p -> p.getNume().startsWith("A");
Predicate<Persoana> criteriu = adult.and(numeA.negate());
```

Metode utile: and, or, negate.

11.6 Consumer

```
Consumer<String> c = s -> System.out.println(s.toUpperCase());
c.accept("java"); // JAVA
```

andThen leagă acțiuni.

11.7 Function

```
Function<Integer, Integer> f = x -> x * x;
Function<Integer, Integer> g = x -> 2 * x;
System.out.println(f.compose(g).apply(2)); // f(g(2)) = 16
System.out.println(f.andThen(g).apply(2)); // g(f(2)) = 8
```

11.8 Supplier

```
Supplier<Integer> s = () -> 10;
System.out.println(s.get()); // 10
```

11.9 Method references

Formă	Exemplu
Class::staticMethod	Math::sin
Class::instanceMethod	String::substring
obj::instanceMethod	p::getNume
Class::new	Persoana::new

```
Consumer<String> c1 = s -> System.out.println(s);
Consumer<String> c2 = System.out::println;
```


11.10 Variabile efectiv finale

```
int prag = 30;  
Predicate<Persoana> p = pers -> pers.getVarsta() >= prag;  
// prag = 40; // ar produce eroare
```

Variabilele locale folosite într-o lambda trebuie să fie finale sau efectiv finale.

Capitolul 12 Stream-uri

12.1 Ce este un stream

Un stream este un flux de elemente dintr-o sursă de date, folosit pentru prelucrare declarativă.

Important

Stream-ul nu este colecție și nu stochează elemente. El procesează elementele din sursă.

Etape:

1. creare stream;
2. operații intermediare;
3. operație terminală.

12.2 Creare stream

```
list.stream();  
Stream.of("a", "b", "c");  
Stream.of(arrayObiecte);  
Arrays.stream(arrayPrimitive);  
Stream.generate(() -> Math.random()).limit(5);  
Stream.iterate(1, x -> x * 2).limit(5);
```

Capcană de examen

```
int[] v = {1, 2, 3};  
System.out.println(Stream.of(v).count()); // 1  
System.out.println(Arrays.stream(v).count()); // 3
```

12.3 Operații intermediare

Operație	Sens
filter	Selectează elementele care respectă un Predicate.
map	Transformă fiecare element într-un alt element.
flatMap	Transformă fiecare element într-un stream și aplatizează.
sorted	Sortează.
distinct	Elimină duplicate folosind equals/hashCode.

limit

Ia cel mult primele n elemente.

```
lp.stream()
  .filter(p -> p.getVarsta() >= 30)
  .sorted(Comparator.comparing(Persoana::getNum))
  .map(Persoana::getNum)
  .forEach(System.out::println);
```

Reține

Operațiile intermediare sunt lazy: nu se execută până nu apare o operație terminală.

12.4 map vs flatMap

```
List<String> linii = Arrays.asList("ana are", "mere bune");

linii.stream()
  .map(l -> l.split(" ")); // Stream<String[]>

linii.stream()
  .map(l -> l.split(" "))
  .flatMap(t -> Arrays.stream(t)) // Stream<String>
  .forEach(System.out::println);
```

12.5 Operații terminale

Operație	Sens
forEach	Execută o acțiune.
count	Numără elementele.
min/max	Găsește minim/maxim după comparator.
reduce	Reduce la o singură valoare.
collect	Colectează în listă, set, map, string, statistici.

12.6 reduce

```
int suma = Stream.of(1, 2, 3, 4)
  .reduce(0, (a, b) -> a + b);
System.out.println(suma); // 10
```

12.7 collect și Collectors

```
List<Persoana> bogati = lp.stream()
  .filter(p -> p.getSalariu() >= 3000)
  .collect(Collectors.toList());

String nume = lp.stream()
  .map(Persoana::getNum)
```

```
.collect(Collectors.joining(", "));

Map<Integer, List<Persoana>> dupaVarsta = lp.stream()
    .collect(Collectors.groupingBy(Persoana::getVarsta));

Map<Integer, Double> salariiPeVarsta = lp.stream()
    .collect(Collectors.groupingBy(
        Persoana::getVarsta,
        Collectors.summingDouble(Persoana::getSalariu)
    ));
```

12.8 Capcane la stream-uri

- Un stream se consumă o singură dată.
- Fără operație terminală, pipeline-ul nu pornește.
- `sorted()` cere `Comparable`; altfel folosești `Comparator`.
- `distinct()` cere `equals/hashCode` corecte.
- `Stream.of(int[])` are un singur element: tabloul.

Capitolul 13 Fire de executare și socket-uri

13.1 Proces vs thread

Un proces este un program în execuție. Un thread este unitatea care execută instrucțiuni în cadrul procesului.

- fiecare thread are stack propriu;
- heap-ul este comun thread-urilor din același proces;
- obiectele partajate pot crea probleme de sincronizare.

13.2 Crearea thread-urilor

Extindere Thread:

```
class Fir extends Thread {  
    public void run() {  
        System.out.println("thread");  
    }  
}  
new Fir().start();
```

Implementare Runnable:

```
class Fir implements Runnable {  
    public void run() {  
        System.out.println("thread");  
    }  
}  
Thread t = new Thread(new Fir());  
t.start();
```

13.3 start vs run

Important

start() pornește un thread nou și invocă run() pe acel thread. run() apelat direct este doar o metodă obișnuită, executată în firul curent.

```
Thread t = new Thread(() -> System.out.print("A "));  
t.start();  
System.out.print("B ");  
// output posibil: A B sau B A
```

13.4 join, sleep, interrupt, volatile

<code>sleep(ms)</code>	Pauză pentru firul curent.
<code>join()</code>	Firul curent așteaptă terminarea altui fir.
<code>interrupt()</code>	Semnalizează întreruperea unui fir.
<code>volatile</code>	Obligă citirea valorii actualizate între thread-uri.

```
t.start();  
t.join();  
System.out.println("ădup terminarea lui t");
```

13.5 synchronized

```
class Counter {  
    private long count;  
    public synchronized void add() {  
        count++;  
    }  
}
```

`count++` nu este atomic. Are nevoie de sincronizare dacă mai multe fire modifică același contor.

```
public void add() {  
    synchronized (this) {  
        count++;  
    }  
}
```

13.6 wait, notify, notifyAll

- `wait()` suspendă thread-ul și eliberează lock-ul;
- `notify()` trezește un thread care așteaptă;
- `notifyAll()` trezește toate thread-urile care așteaptă;
- se folosesc în context sincronizat.

```
public synchronized void pune(int x) throws InterruptedException {  
    while (plin()) wait();  
    adauga(x);  
    notify();  
}
```

13.7 Socket-uri

Model client-server:

- serverul pornește primul;
- clientul se conectează la adresă + port;
- comunicarea se face prin fluxuri de intrare/ieșire.

Server:

```
ServerSocket ss = new ServerSocket(1234);  
Socket cs = ss.accept(); // șăateapt client  
DataInputStream in = new DataInputStream(cs.getInputStream());  
DataOutputStream out = new DataOutputStream(cs.getOutputStream());
```

Client:

```
Socket cs = new Socket("localhost", 1234);  
DataInputStream in = new DataInputStream(cs.getInputStream());  
DataOutputStream out = new DataOutputStream(cs.getOutputStream());  
out.writeUTF("Salut");  
String raspuns = in.readUTF();
```

Capitolul 14 Test final de recapitulare

14.1 Exerciții rapide

1. Ce afișează?

```
String a = "x";  
String b = new String("x");  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

Răspuns: false, apoi true.

2. Se compilează?

```
interface I { void f(); }  
class A implements I { void f() {} }
```

Răspuns: Nu. Metoda trebuie să fie public.

3. Ce afișează?

```
class A { int x = 1; void f(){System.out.print("A");} }  
class B extends A { int x = 2; void f(){System.out.print("B");} }  
A ob = new B();  
System.out.print(ob.x);  
ob.f();
```

Răspuns: 1B.

4. Ce afișează?

```
List<Integer> l = Arrays.asList(1, 2, 3, 4, 5);  
l.stream().filter(x -> x % 2 == 1)  
    .map(x -> x * 10)  
    .forEach(x -> System.out.print(x + " "));
```

Răspuns: 10 30 50.

5. Ce afișează?

```
int[] v = {1, 2, 3};  
System.out.println(Stream.of(v).count());  
System.out.println(Arrays.stream(v).count());
```

Răspuns: 1, apoi 3.

6. Ce problemă are codul?

```
DataOutputStream out = new DataOutputStream(new FileOutputStream("x.bin"));  
out.writeInt(10);  
out.writeDouble(2.5);  
DataInputStream in = new DataInputStream(new FileInputStream("x.bin"));  
double d = in.readDouble();  
int x = in.readInt();
```

Răspuns: Citește în altă ordine decât a scris.

7. Ce se întâmplă?

```
Stream<Integer> s = Stream.of(1, 2, 3);  
s.count();  
s.count();
```

Răspuns: A doua folosire produce eroare; stream-ul este deja consumat.

8. Ce face join()? **Răspuns:** Firul curent așteaptă terminarea altui fir.9. Ce se serializează: static, transient, private? **Răspuns:** private da, static nu, transient nu.10. Ce face PreparedStatement? **Răspuns:** Execută SQL parametrizat cu ?, mai sigur și mai potrivit pentru valori externe.

14.2 Checklist final înainte de examen

- main corect și de ce este static;
- primitive vs referințe;
- == vs equals;
- stack vs heap;
- constructori și this/super;
- overriding vs overloading;
- câmpuri vs metode statice vs metode non-statice;
- clase abstracte;
- interfețe și metode public;
- Comparable/Comparator;
- enum: name, ordinal, values;
- checked vs unchecked;
- throw vs throws;
- finally;
- fișiere: read() == -1, readLine() == null;
- DataInputStream/DataOutputStream;
- serializare: Serializable, transient, serialVersionUID;
- colecții: List, Set, Map;
- HashSet/TreeSet;
- lambda și interfețe funcționale;
- Predicate/Consumer/Function/Supplier;
- stream: filter, map, flatMap, collect;
- thread: start vs run, join, synchronized;
- socket: ServerSocket, Socket, accept.

Asocieri rapide de memorat

Dacă vezi...	Gândește-te la...
<code>new String("x")</code>	obiect nou, == probabil false cu literalul.
<code>String s;</code>	nu modifică s; trebuie asignare.
<code>s.toUpperCase();</code>	
<code>A ob = new B(); ob.f();</code>	metodă non-statică: tip real.
<code>A ob = new B(); ob.x</code>	câmp: tip declarat.
<code>static</code>	apartține clasei.
<code>final int[]</code>	nu poți schimba referința, dar poți modifica elementele.
<code>catch(Exception) înainte de catch(IOException)</code>	eroare, handler inaccesibil.
<code>read()</code>	-1 la final.
<code>readLine()</code>	null la final.
<code>Serializable</code>	marker, serializare automată.
<code>Externalizable</code>	control manual, constructor fără parametri.
<code>HashSet</code>	hashCode + equals.
<code>TreeSet</code>	compareTo/Comparator.
<code>Predicate</code>	test(), returnează boolean.
<code>Consumer</code>	accept(), nu returnează nimic.
<code>Function</code>	apply(), transformă T în R.
<code>Supplier</code>	get(), nu primește parametri.
<code>filter</code>	selectează.
<code>map</code>	transformă.
<code>flatMap</code>	aplatizează.
<code>start()</code>	thread nou.
<code>run()</code>	metodă normală dacă o apelezi direct.
<code>synchronized</code>	excludere reciprocă.
<code>accept() la server</code>	blochează până vine client.